

PropAMMRouter

Security Audit — Initial Audit Response

LambdaClass

Initial Audit Response: 21 August 2026

Table of Contents

Summary

Issues

Issue A: Swap Functions May Ignore a Better Swap Quote

Issue B: Fee-Bearing Swaps May Report More Output Than the Recipient Receives

Issue C: Selected Venue Fee Swaps Attempt Venues That Cannot Meet the Gross Minimum

General Comments

Documentation and Code Comments

Scope

Verification

Note on the fork-test commit

Overview

Project

PropAMMRouter

Audit date

July 15 – July 22, 2026

Audit report delivered

July 24, 2026

Audit revision

e413bc931318fcd72a18c892a74db993145d5c98 (branch `contracts/audit/base`)

Verification revision

38b3b4254218d232724a3d5eff862c01b0dc4d1f (branch [contracts/audit/fixes](#))

Response date

August 21, 2026

Summary

The audit identified 3 issues (A, B and C). Here's a table with the resolution for each of them.

Category	Count
Fixed (with PR)	2 issues (B, C)
Documented as intended design (auditor's alternative remediation)	1 issue (A)

Each finding is addressed below with links to the relevant PRs. All changes are present on the `contracts/audit/fixes` branch, which is based on the `contracts/audit/base`, audited branch.

Additionally, the general comment on stale code comments is addressed below.

Change	Issue	Kind
#86	A	Documentation
#81	B	Fix + tests
#79	C	Fix
45534cc9	–	Test-harness repair (see note below)

Issues

Issue A: Swap Functions May Ignore a Better Swap Quote

Severity

Medium

Status

Documented as intended design (remediation option 2)

PR

[#86](#)

Finding

quoteV1 calls `_pickBestVenue`, which compares all whitelisted propAMMs and the Uniswap fallback. swapV1 and swapWithFeeV1 call `_pickBestPropAMM`, which excludes Uniswap whenever a propAMM quote satisfies the caller's minimum output. Under identical state, quoteV1 can report Uniswap as best while swapV1 executes through a less favorable propAMM.

Response

We confirm the divergence and adopt the auditor's second remediation option: swapV1 is a **propAMM-preferred** swap. That is: Uniswap is only a fallback mechanism designed to be used on blocks where no propamm-enabled builder is selected, not another propamm to quote for best price. In practice, about 1% of the swaps have a better price by Uniswap, not covering the gas costs of requoting it for every swap.

This design is deliberate and predates the audit. It was introduced by [#71](#) (*perf(contract): remove Uniswap quote in requoting swap functions*), which is the audited revision itself.

From [docs/gas_report.md](#), quoting Uniswap as a third venue on top of Fermi + Kipseli:

Venues quoted	Gas	Δ vs. previous
Fermi + Kipseli (2)	331,892	–
Fermi + Kipseli + Uniswap (3)	439,539	+107,647

Including Uniswap requoting would add the 65% gas cost of a full Uniswap swap for all swaps. In practice, Uniswap has a better price in less 1% of the trades. For those cases, the best propAMM is typically still over the `amountOutMin` and the spread is very tight. In theory, in a trade where Uniswap was significantly better and no propAMM was able to give a better quote than the `amountOutMin`, Uniswap would still be used as a fallback, therefore still selecting the best price, without the additional requoting.

Resolution

PR #86 documents this decision:

- `IPropAMMRouter.swapV1 NatSpec`: *"Only the propAMMs are quoted for selection (quoting the fallback on-chain costs gas), so the fallback is picked only when no propAMM can be priced or the best one is below amountOutMin."*
- `IPropAMMRouter.swapWithFeeV1 NatSpec`: same, against the grossed-up `amountOutMin`.
- `IPropAMMRouter.quoteV1 NatSpec`: *"Unlike swapV1, this includes the fallback, so venue is the price to beat rather than the venue swapV1 would pick."*

- README.md: the swapV1 and swapViaSelectedVenuesV1 bullets now say the router routes to Uniswap V3 when the propAMM quote is below amountOutMin, rather than implying a best-of-all-venues selection.

On the request for a matching quote function

The audit requested that a separate quote function be provided whose venue-selection behavior exactly matches the corresponding swap function. That path already exists and needs no new API surface:

```
address[] memory propAMMs = router.getWhitelistedVenues();           // excludes the
fallback
(uint256 quote, address venue) = router.quoteSelectedVenuesV1(propAMMs, tokenIn,
tokenOut, amountIn);
```

quoteSelectedVenuesV1 routes through `_pickBestVenueFrom`, and swapV1 routes through `_pickBestPropAMM`. Both iterate the same candidate set (`getWhitelistedVenues()` returns exactly the set `_pickBestPropAMM` enumerates via `whitelistedVenueCount/whitelistedVenueAt`), both skip reverting venues in individual `try/catch` blocks, and both resolve the quote through the same private `_quoteVenue` body. The only caller-side step is the `amountOutMin` comparison that decides the fallback, which is a plain numeric check on the returned quote.

Integrators wanting the true just-in-time, best-of-all-venues execution may use `swapViaSelectedVenuesV1(venues, ...)` with the `SwapRouter02` address included in `venues`. This makes the explicit choice of quoting uniswap and paying the extra ~108k gas cost.

We consider the finding addressed: the router's advertised behavior now matches its implementation, the price-to-beat semantics of `quoteV1` are stated on the function itself, and callers who want best-of-all-venues selection have a documented path that do exactly that.

Issue B: Fee-Bearing Swaps May Report More Output Than the Recipient Receives

Severity

Medium

Status

Fixed

PR

[#81](#)

Finding

`FrontendFees._skimAndDisburse` (`src/libraries/FrontendFees.sol#L59`) computes the nominal fee and net amounts, transfers the net to the recipient, and returns the nominal net without measuring the recipient's balance change. `_coreSwap` enforces `grossMin` against the router's balance delta, so it cannot see a deduction that happens during the subsequent router-to-recipient transfer. With a fee-on-transfer or otherwise short-crediting `tokenOut`, the swap can succeed and report the nominal net even though the recipient received less than `amountOutMin`.

Resolution

PR #81 moves the slippage check after the delivery of the token to the recipient. `_skimAndDisburse` now takes `amountOutMin` and, for the ERC-20 net leg, snapshots the recipient's balance, performs the transfer, and recomputes net as the measured delta:

```

uint256 prevBalance = IERC20(tokenOut).balanceOf(recipient);
if (net > 0) {
    IERC20(tokenOut).safeTransfer(recipient, net);
}
net = IERC20(tokenOut).balanceOf(recipient) - prevBalance;
...
require(net >= amountOutMin, InsufficientOutput(amountOutMin, net));

```

The net is recalculated by using the balance delta after the transfer. This has impact in all three areas described in the audit:

- **Slippage check.** The function now requires the measured delta is greater than amountOutMin. If the result is too low, the router now reverts with `InsufficientOutput(amountOutMin, actualReceipt)`.
- **Return value.** `_skimAndDisburse` returns the measured delta, so `swapWithFeeV1`, `swapViaVenueWithFeeV1`, and `swapViaSelectedVenuesWithFeeV1` all return what the recipient actually holds.
- **Event.** All three entry points pass that same measured value into `_emitSwapped`, so `Swapped.amountOut` no longer overstating the recipient's output.

Two legs remain nominal, both documented in the NatSpec:

1. **Native ETH.** When `tokenOut == ETH_SENTINEL` the payout is a raw `call{value: net}`; ETH credits exactly and has no transfer hook, so there is nothing to measure. The branch structure was reorganized so the ETH and ERC-20 paths are separate, with the `net > 0` zero-value-transfer skip preserved inside each.
2. **Fees.** `FrontendFeeCharged` reports what the router sent to `fee.recipient`. A short-credit on that leg is the fee recipient's concern, not a breach of the caller's slippage guarantee, and measuring it would add a second pair of `balanceOf` calls to every fee-bearing swap.

Scope note

The non-fee entry points (`swapV1`, `swapViaVenueV1`, `swapViaSelectedVenuesV1`) were never affected. They pass the end recipient straight into `_coreSwap`, which snapshots recipient's balance before execution and enforces `amountOutMin` against that delta directly. Only the fee-bearing paths route output to `address(this)` first, which is what created the unmeasured second hop.

Tests

PR #81 adds `test_swapWithFee_fotOutBelowMinReverts`, which drives a 1%-fee-on-transfer `tokenOut` through `swapWithFeeV1` with `netMin` set one wei above the recipient's true receipt and asserts the exact `InsufficientOutput(netMin, received)` revert - i.e. the case that previously settled silently. `test_swapWithFee_feeOnTransferTokenOut` was updated to assert that the returned `amountOut` equals the recipient's measured receipt rather than the nominal net. The existing `testFuzz_swapWithFee_netNeverBelowMin` fuzz case (256 runs over `feeBps`, `netMin`, and `delivered`) continues to pass, confirming the new check does not reject well-behaved tokens at any fee tier.

Issue C: Selected Venue Fee Swaps Attempt Venues That Cannot Meet the Gross Minimum

Severity

Low

Status

Fixed

PR

[#79](#)

Finding

swapViaSelectedVenuesWithFeeV1 (src/PropAMMRouter.sol#L263) computed grossMin for execution but gated venue selection on the *net* amountOutMin. A venue quoting between amountOutMin and grossMin therefore passed selection, then failed execution against grossMin, burning gas on a rolled-back attempt before _coreSwap engaged the Uniswap fallback – and if insufficient gas remained for the fallback, the whole swap reverted.

Resolution

PR #79 applies the auditor's recommendation directly, using grossMin consistently for selection and execution:

```
-         if (venue == address(0) || bestQuote < amountOutMin) {  
+         if (venue == address(0) || bestQuote < grossMin) {  
            venue = fallbackSwapRouter;  
        }
```

swapViaSelectedVenuesWithFeeV1 now behaves identically to swapWithFeeV1, which already compared its quote against grossMin: a venue that cannot cover the user's minimum *plus* the frontend fee is never attempted, and the router routes straight to the fallback instead. The wasted venue call, the divergence between the two fee-bearing best-venue paths, and the gas-exhaustion path to a spurious revert are all eliminated.

Test coverage note

No dedicated regression test was added for the selected-venues variant. The equivalent invariant on the sibling path is covered by test_swapWithFee_revertsQuoteBelowGrossMin, which asserts that a quote below grossMin produces InsufficientOutput(grossMin, delivered) rather than settling short, and test_swapViaSelectedVenuesWithFee_takesFee covers the happy path through the fixed function. We are happy to add a swapViaSelectedVenuesWithFeeV1-specific case asserting the selection-time routing decision if the verification review would like one.

General Comments

Documentation and Code Comments

Comment

"Some comments describe intended behavior that differs from the implementation. We recommend updating stale or conflicting comments."

Status

Addressed

PR

[#86](#)

The main instance was the one underlying Issue A: the swapV1 family was described as best-venue routing while it selected among propAMMs only, and the README.md swapV1 bullet claimed a pre-pull

InsufficientOutput revert on a below-minimum quote when the implementation routes to the fallback instead. PR #86 corrects both, in the interface NatSpec and in the README. Beyond that specific case, we treat comment accuracy as an ongoing review obligation rather than a one-off change; if the verification review has further specific instances in mind, we will fix them directly.

Scope

Comment

"We recommend conducting a focused review for the off-chain infrastructure in the future."

Status

Acknowledged

Agreed. The off-chain quoting path is a good candidate for a review. The `titan_getPammStateOverrides` state-override flow that makes propAMM quotes available between blocks, and the SDKs that consume it will be scoped in a separate engagement.

Verification

The changes compose cleanly on top of the audited revision. Against the combined tree (base + #79 + #86 + 45534cc9 + #81):

- `forge build` succeeds; the only diagnostics are pre-existing lints (`block.timestamp` in the deadline check, unchecked transfer return values in test helpers).
- `forge test`, with `RPC_URL` pointed at a mainnet endpoint, reports **89 passed, 0 failed, 1 skipped** across all 6 suites (90 tests total).
- The single skip is `test_swapViaVenueV1NewKipseli`, which is `vm.skip(true)` in the committed source with the reason inline: it rolls the fork back to a historical block and therefore needs an archive RPC. This predates the audit and is unrelated to any finding.

Note on the fork-test commit

`contracts/audit/fixes` carries one commit that does not correspond to an audit finding, and we call it out here so the diff between the audited revision and the verification revision holds no surprises.

45534cc9 – test(contracts): refresh Fermi price target in fork test. This is a cherry-pick of d13803e3 from `fix/fork-test-fermi-price-target`, and it touches `test/PropAMMRouterForkTests.t.sol` only – no contract source. We picked it onto the `fixes` branch so that the branch under verification has a green suite; without it, `test_swapViaVenueV1Fermi` fails on the audited revision for a reason that has nothing to do with Issues A–C:

- Fermi's `PrioUpdateRegistry` scopes a pricing lane by the account that calls `getState`. The test published its price update to the lane for `0x26e5A56f . . .`, but Fermi has since rotated its live pricing contract to `0xD0fD9FC7 . . .`, so the update landed on a lane nobody reads and the venue found its own lane empty.
- `getState` then reverts with `0x666a2814`, which propagates out of the Fermi quote and fails the test. Reproduced on the pre-fix tree: `[FAIL: fj() test_swapViaVenueV1Fermi() – fj(` is how Forge renders the ASCII bytes of that selector.
- The commit repoints `priceTarget` at the current pricing contract. The lane index and price slot are unchanged; only the target rotated. The comment above it was also corrected – it previously claimed

the target came from the referenced mainnet updater tx, which in fact used a third address (0xe514a3c4 . . .) – and now records how to refresh the address from the getState caller in forge test -vvvv the next time Fermi redeploys.

The dependence on a live, rotatable mainnet address is a property of the fork test, not of the router. We mention it because it means the fork lane can go red again on a future redeploy without any change to the contracts, and a verification review that hits 0x666a2814 should read it as a stale test fixture rather than a regression.